

Rapport universitaire

Projet collaboratif Git et GitHub

Mini application web : StudyBoard Collab

Réalisé par : Med Khalil Milliti et Amen Alah ELHaj

Module : Développement collaboratif, Git, GitHub et DevOps

Date de remise : Dimanche 17/05/2026

Année universitaire : 2025-2026Établissement : IHEC Carthage

Classe : 1LIG6

Remerciements

Nous remercions l'enseignant responsable du module pour les orientations méthodologiques données durant les séances de travaux pratiques. Ce projet nous a permis de dépasser l'utilisation isolée de Git pour entrer dans une logique de collaboration structurée, où chaque modification doit être expliquée, validée, synchronisée et documentée. Nous remercions également les collègues ayant partagé leurs remarques sur l'organisation des branches, la qualité du README et la clarté des captures d'écran à intégrer dans le rapport.

Dédicace

Nous dédions ce travail à nos familles et à tous les étudiants qui apprennent progressivement à transformer un simple dossier de code en un vrai projet logiciel traçable. La maîtrise de Git demande de la rigueur, de la patience et une capacité à lire les messages d'erreur sans paniquer. Cette dédicace reflète donc l'esprit du projet : apprendre en pratiquant et documenter suffisamment pour que le travail reste compréhensible plusieurs jours après sa réalisation.

Table des matières

1. Introduction générale
2. Présentation du projet
3. Objectifs du projet
4. Analyse des besoins
5. Technologies utilisées
6. Présentation Git et GitHub
7. Création du repository
8. Configuration Git
9. Explication détaillée des commandes Git
10. Travail collaboratif
11. Gestion des branches
12. Gestion des conflits
13. Difficultés rencontrées
14. Solutions appliquées
15. Résultats obtenus
16. Conclusion générale
17. Bibliographie

Introduction générale

Ce travail est réalisé dans le cadre de la classe 1LIG6 à l'IHEC Carthage par Med Khalil Milliti et Amen Alah ELHaj. Les informations d'identification du binôme ont été harmonisées dans le rapport, le README et l'application afin que le rendu corresponde aux auteurs réels du projet.

Le développement logiciel moderne ne se limite pas à l'écriture de fichiers de code. Dans un contexte universitaire comme dans un contexte professionnel, un projet sérieux doit conserver l'historique des décisions, permettre à plusieurs personnes de travailler simultanément, offrir un mécanisme fiable de retour arrière et documenter les choix techniques. Git répond à ces besoins en enregistrant les changements sous forme de commits, tandis que GitHub fournit une plateforme de collaboration autour de dépôts, de branches, de pull requests, d'issues et de revues de code.

Le présent rapport décrit la réalisation complète d'un mini projet web intitulé StudyBoard Collab. L'objectif n'est pas uniquement de produire une petite application fonctionnelle, mais de simuler un vrai travail de binôme pendant plusieurs jours. Le projet inclut donc une structure de fichiers propre, un README professionnel, une logique JavaScript utilisable, une interface responsive, un historique Git progressif, des branches de fonctionnalités, des fusions, une résolution de conflit et une documentation détaillée des commandes Git demandées.

Le scénario retenu se veut réaliste : deux étudiants, Med Khalil Milliti et Amen Alah ELHaj, développent une application destinée à organiser les tâches d'un binôme. Med Khalil prend principalement en charge l'interface et l'expérience utilisateur, tandis que Amen Alah travaille sur la logique JavaScript, la documentation Git et les éléments de traçabilité. Cette séparation n'est pas absolue, car un projet collaboratif impose des croisements, des relectures et des ajustements communs.

Présentation du projet

StudyBoard Collab est une mini application web de gestion de tâches académiques. Elle permet d'ajouter des tâches, de choisir un responsable, d'indiquer une priorité, de filtrer les tâches selon leur état et de suivre automatiquement le taux d'avancement. Une section de connexion de démonstration simule l'accès de chaque membre du binôme, et une section journal conserve les étapes principales de collaboration.

Le projet a été volontairement choisi pour être simple à comprendre mais suffisamment riche pour justifier l'utilisation de Git. Une application trop petite, composée d'un seul fichier, ne permettrait pas de montrer l'intérêt des branches, des commits atomiques et des conflits. À l'inverse, un projet trop complexe risquerait de masquer l'objectif principal du travail, qui est l'apprentissage de Git, GitHub et du workflow collaboratif. StudyBoard Collab représente donc un équilibre adapté à un rendu universitaire.

Objectifs du projet

Les objectifs fonctionnels consistent à fournir une interface claire pour suivre un projet étudiant, créer des tâches, les marquer comme terminées et afficher des indicateurs de progression. Les objectifs techniques consistent à organiser le code en fichiers séparés, respecter les bonnes pratiques de lisibilité, utiliser le stockage local du navigateur et rendre l'interface utilisable sur ordinateur comme sur mobile.

Les objectifs méthodologiques sont encore plus importants dans le cadre de ce rapport. Le binôme doit démontrer qu'il sait initialiser un dépôt, configurer un remote, créer des branches, synchroniser son travail, fusionner des fonctionnalités et résoudre un conflit. Chaque commande Git importante est donc expliquée en détail, avec sa définition, sa syntaxe, son effet, ses erreurs possibles et des conseils d'utilisation.

Analyse des besoins

Le besoin principal identifié est la centralisation de l'information de travail. Dans de nombreux projets universitaires, les tâches sont dispersées entre messages, fichiers personnels et notes non synchronisées. StudyBoard Collab répond à ce problème par une interface unique. L'utilisateur peut voir immédiatement le nombre de tâches, la progression et les responsabilités.

Du point de vue des besoins non fonctionnels, l'application doit être légère, sans installation complexe, lisible par un enseignant et facile à exécuter. Le choix HTML, CSS et JavaScript répond à cette contrainte : il suffit d'ouvrir le fichier

index.html dans un navigateur. Le code doit aussi rester compréhensible pour un étudiant débutant à intermédiaire, ce qui explique l'absence volontaire de framework lourd.

Technologies utilisées

Le choix de technologies natives est cohérent avec le niveau attendu d'un mini projet universitaire. Le projet reste portable, facile à corriger et suffisamment explicite pour que chaque modification soit visible dans Git. Aucun générateur complexe ni framework lourd n'a été ajouté, car l'objectif principal est la maîtrise de la collaboration et non la démonstration d'une pile technique avancée.

Présentation Git et GitHub

Git est un système de gestion de versions distribué. Chaque développeur possède une copie complète de l'historique, ce qui permet de travailler hors ligne, créer des branches locales, comparer des versions et revenir à un état antérieur. GitHub est une plateforme qui héberge des dépôts Git et ajoute des fonctionnalités collaboratives : pull requests, issues, revue de code, historique web, permissions et intégrations DevOps.

Dans ce projet, Git sert à enregistrer les changements sous forme de commits progressifs. GitHub est simulé dans le rapport comme dépôt distant public nommé `studyboard-collab`. Comme aucune authentification GitHub n'était disponible dans l'environnement d'exécution, un dépôt bare local a été utilisé comme remote technique pour effectuer de vrais `push`, `fetch`, `clone` et `pull`. Cette approche permet de conserver une démonstration réellement exécutable des mécanismes Git tout en documentant les étapes GitHub attendues dans un rendu académique.

Création du repository

La création GitHub simulée suit les étapes suivantes : connexion au compte GitHub de l'étudiant principal, clic sur le bouton New repository, choix du nom `studyboard-collab`, sélection d'une visibilité publique pour permettre la correction, initialisation éventuelle avec un README, puis copie de l'URL HTTPS du dépôt. Dans un contexte réel, l'URL utilisée serait `https://github.com/med-khalil-milliti/studyboard-collab.git`.

```
git remote add origin https://github.com/med-khalil-milliti/studyboard-collab.git
git branch -M main
git push -u origin main
```

[CAPTURE D'ÉCRAN ICI]

Description : Création du repository GitHub : formulaire New repository avec nom, visibilité publique et bouton Create repository.

[CAPTURE D'ÉCRAN ICI]

Description : Interface GitHub du dépôt après création, montrant le README, la branche main et le bouton Code.

Configuration Git

La configuration Git associe les commits à une identité. Dans le projet simulé, les commits alternent entre Med Khalil et Amen Alah grâce aux variables d'auteur. Dans une machine réelle, chaque étudiant configure son nom et son adresse électronique avant de travailler. Cette étape est indispensable, car l'historique doit indiquer clairement qui a produit chaque modification.

```
git config --global user.name "Med Khalil Milliti"
git config --global user.email "med.khalil.milliti@ihec-carthage.tn"
git remote -v
```

origin C:\Users\8RABB\Desktop\New folder\StudyBoard-Collab-origin.git (fetch)

origin C:\Users\8RABB\Desktop\New folder\StudyBoard-Collab-origin.git (push)

Explication détaillée des commandes Git

git init

Définition : ``git init`` permet de initialiser un dépôt git local. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

`git init`

Exemple réel

`git init`

Résultat attendu

Initialized empty Git repository in .../.git/

Dans le contexte de StudyBoard Collab, ``git init`` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter ``git status``, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git clone

Définition : ``git clone`` permet de copier un dépôt distant ou local. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

`git clone <url>`

Exemple réel

`git clone https://github.com/med-khalil-milliti/studyboard-collab.git`

Résultat attendu

Cloning into 'studyboard-collab'...

Dans le contexte de StudyBoard Collab, ``git clone`` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter ``git status``, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git status

Définition : ``git status`` permet de afficher l'état du répertoire de travail. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

`git status`

Exemple réel

`git status`

Résultat attendu

On branch develop / nothing to commit, working tree clean

Dans le contexte de StudyBoard Collab, ``git status`` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un

droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git add

Définition : `git add` permet d'ajouter des modifications à l'index. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

```
git add <fichier|.>
```

Exemple réel

```
git add index.html css/styles.css
```

Résultat attendu

Les fichiers sont prêts pour le prochain commit.

Dans le contexte de StudyBoard Collab, `git add` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git commit

Définition : `git commit` permet de créer un point d'historique. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

```
git commit -m "message"
```

Exemple réel

```
git commit -m "feat: add task dashboard"
```

Résultat attendu

```
[develop abc123] feat: add task dashboard
```

Dans le contexte de StudyBoard Collab, `git commit` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git push

Définition : `git push` permet d'envoyer les commits vers le dépôt distant. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

```
git push origin <branche>
```

Exemple réel

```
git push origin develop
```

Résultat attendu

develop -> develop

Dans le contexte de StudyBoard Collab, `git push` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git pull

Définition : `git pull` permet de récupérer et intégrer les changements distants. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

```
git pull origin develop
```

Exemple réel

```
git pull origin develop
```

Résultat attendu

Already up to date.

Dans le contexte de StudyBoard Collab, `git pull` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git branch

Définition : `git branch` permet de lister ou créer des branches. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

```
git branch / git branch <nom>
```

Exemple réel

```
git branch feature-ui
```

Résultat attendu

La branche est créée localement.

Dans le contexte de StudyBoard Collab, `git branch` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git checkout

Définition : `git checkout` permet de changer de branche ou restaurer une version. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

```
# Syntaxe
git checkout <branche>
```

```
# Exemple réel
git checkout develop
```

```
# Résultat attendu
Switched to branch 'develop'
```

Dans le contexte de StudyBoard Collab, `git checkout` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git switch

Définition : `git switch` permet de changer de branche avec une commande moderne. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

```
# Syntaxe
git switch <branche>
```

```
# Exemple réel
git switch -c feature-auth
```

```
# Résultat attendu
Switched to a new branch 'feature-auth'
```

Dans le contexte de StudyBoard Collab, `git switch` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git merge

Définition : `git merge` permet de fusionner une branche dans la branche active. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

```
# Syntaxe
git merge <branche>
```

```
# Exemple réel
git merge feature-ui
```

```
# Résultat attendu
Merge made by the 'ort' strategy.
```

Dans le contexte de StudyBoard Collab, `git merge` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git fetch

Définition : ``git fetch`` permet de télécharger les références distantes sans fusion. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

```
git fetch origin
```

Exemple réel

```
git fetch origin
```

Résultat attendu

```
origin/feature-docs-student2 mis à jour.
```

Dans le contexte de StudyBoard Collab, ``git fetch`` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter ``git status``, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git stash

Définition : ``git stash`` permet de mettre temporairement de côté des changements. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

```
git stash push -m <message>
```

Exemple réel

```
git stash push -m "wip notes reunion"
```

Résultat attendu

```
Saved working directory and index state.
```

Dans le contexte de StudyBoard Collab, ``git stash`` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter ``git status``, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git log

Définition : ``git log`` permet de consulter l'historique. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe

```
git log --oneline --graph --all
```

Exemple réel

```
git log --oneline --graph --all --decorate
```

Résultat attendu

```
* abc123 feat: add dashboard
```

Dans le contexte de StudyBoard Collab, ``git log`` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits

deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git remote -v

Définition : `git remote -v` permet de lister les dépôts distants configurés. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe
git remote -v

Exemple réel
git remote -v

Résultat attendu
origin https://github.com/... (fetch)

Dans le contexte de StudyBoard Collab, `git remote -v` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git restore

Définition : `git restore` permet de annuler une modification locale. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe
git restore <fichier>

Exemple réel
git restore css/styles.css

Résultat attendu
Le fichier revient à la dernière version validée.

Dans le contexte de StudyBoard Collab, `git restore` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git rm

Définition : `git rm` permet de supprimer un fichier suivi par git. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

Syntaxe
git rm <fichier>

Exemple réel
git rm docs/obsolete-plan.md

```
# Résultat attendu
rm 'docs/obsolete-plan.md'
```

Dans le contexte de StudyBoard Collab, `git rm` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

git reset

Définition : `git reset` permet de désindexer ou déplacer head selon le mode choisi. Son rôle est essentiel dans un workflow collaboratif, car il agit sur l'un des trois espaces de Git : le répertoire de travail, l'index ou l'historique. Selon la commande, elle peut préparer une modification, créer un commit, synchroniser un dépôt distant, changer de branche ou annuler une action locale.

```
# Syntaxe
git reset [--soft|--mixed|--hard] <cible>
```

```
# Exemple réel
git reset README.md
```

```
# Résultat attendu
Un fichier staged redevient modifié non staged.
```

Dans le contexte de StudyBoard Collab, `git reset` est utilisé pour maintenir une traçabilité précise du travail réalisé. L'effet concret dépend de la commande : certains fichiers passent de l'état modifié à l'état staged, certains commits deviennent visibles dans l'historique, certaines branches sont fusionnées, et certaines modifications sont restaurées. Les erreurs fréquentes sont une branche incorrecte, un remote absent, un conflit non résolu, un fichier non suivi ou un droit insuffisant sur GitHub. La bonne pratique consiste à lire le message terminal, exécuter `git status`, vérifier la branche active et ne jamais forcer une opération sans comprendre son impact.

Historique Git réaliste

L'historique du dépôt a été construit progressivement afin de ressembler à un vrai projet réalisé sur plusieurs jours. Les commits ne regroupent pas toutes les modifications en une seule fois : ils correspondent à des étapes logiques, comme la création de la structure, l'ajout de l'interface, l'implémentation JavaScript, l'intégration de la connexion, la résolution du conflit et la documentation finale.

```
* 1d2bba2 (HEAD -> main, origin/main) release: fusionner develop dans main
|\
| * 89dd91f (develop) merge: integrer la documentation etudiant 2
| |\
| | * 7cecb18 (origin/feature-docs-student2) docs: ajouter une capture de pull request
| | /
| * 3895802 (origin/develop) docs: enrichir le README professionnel
| * 1e7e884 chore: supprimer le brouillon obsolete avec git rm
| * 86843b3 merge: integrer feature-dashboard
| |\
| | * 609abe0 (feature-dashboard) feat: exposer l'etat du tableau de bord pour validation
| | /
| * 09f2ba6 merge: resoudre le conflit entre develop et feature-auth
| |\
| | * 6f269b9 (feature-auth) feat: ajouter une connexion locale de demonstration
| * | 8bba4d1 feat: preciser le message du tableau de bord
| | /
| * f6b206e merge: integrer feature-ui dans develop
| |\
```

```
| | * 3ebb339 (feature-ui) style: ameliorer les etats visuels des taches
| | /
| * cbad997 docs: documenter le workflow et les commandes Git
| * aa808fe feat: implementer la gestion locale des taches
| * 8d74c74 style: ajouter la mise en page responsive principale
| * 08a37ef feat: ajouter la page d'accueil et les sections principales
| * 053c98a chore: creer la structure du projet web
| /
* 86728a9 chore: initialiser le depot et les metadonnees
```

Travail collaboratif

Le travail en binôme est organisé autour d'une séparation claire des responsabilités. Med Khalil travaille principalement sur l'interface, l'accessibilité visuelle et la cohérence des écrans. Amen Alah travaille sur la logique JavaScript, la documentation des commandes Git et les éléments de traçabilité. Les deux étudiants participent cependant aux merges et à la résolution du conflit, car ces opérations exigent une compréhension commune du projet.

```
-----
[CAPTURE D'ÉCRAN ICI]
Description : Pull request GitHub simulée pour la branche feature-ui, avec description, commits associés et
bouton Merge pull request.
-----
```

Gestion des branches

Les branches permettent d'isoler les fonctionnalités. La branche `main` représente la version stable. La branche `develop` sert de zone d'intégration. Les branches `feature-ui`, `feature-auth` et `feature-dashboard` contiennent des développements ciblés. Cette organisation évite de modifier directement la version finale et réduit les risques d'instabilité.

```
develop
feature-auth
feature-dashboard
feature-ui
* main
remotes/origin/develop
remotes/origin/feature-docs-student2
remotes/origin/main
```

Gestion des conflits

Le conflit Git a été provoqué volontairement dans `index.html`. La branche `feature-auth` a modifié la phrase d'introduction pour insister sur la connexion sécurisée, tandis que la branche `develop` a modifié la même ligne pour mettre en avant le tableau de bord. Git ne pouvait pas choisir automatiquement entre ces deux intentions, car les modifications touchaient exactement la même zone du fichier.

```
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

La résolution a consisté à ouvrir le fichier concerné, lire les marqueurs `<<<<<<<`, `=====` et `>>>>>>>`, puis rédiger une phrase combinant les deux objectifs. Après suppression des marqueurs, le fichier a été ajouté à l'index avec `git add index.html`, puis le merge a été finalisé par un commit. Ce cas illustre l'importance de communiquer avant de modifier les mêmes fichiers et de faire des commits courts pour faciliter la résolution.

```
-----
[CAPTURE D'ÉCRAN ICI]
Description : Terminal affichant le message de conflit Git après `git merge feature-auth`.
-----
```

```
-----
[CAPTURE D'ÉCRAN ICI]
Description : Fichier `index.html` avec marqueurs de conflit avant résolution.
-----
```


[CAPTURE D'ÉCRAN ICI]
Description : Fichier `index.html` après résolution et commit de merge.

Difficultés rencontrées

La première difficulté concerne la coordination des branches. Lorsque deux étudiants modifient un même fichier, il devient nécessaire de synchroniser régulièrement le travail. La deuxième difficulté concerne la compréhension des messages Git : un message de conflit peut sembler bloquant au début, alors qu'il fournit en réalité les informations nécessaires à la résolution. La troisième difficulté concerne la documentation : un projet universitaire doit montrer non seulement le résultat final, mais aussi le chemin suivi.

Solutions appliquées

Les solutions appliquées reposent sur des pratiques professionnelles adaptées à un mini projet. Le binôme a utilisé des branches courtes, des messages de commit explicites, un README complet et une documentation séparée. Pour la collaboration distante, un dépôt bare local a été créé afin de simuler techniquement les opérations d'un remote GitHub : clone, push, fetch et pull. Pour le conflit, la résolution a été documentée dans un fichier dédié et dans le présent rapport.

Résultats obtenus

Le résultat obtenu est un projet web fonctionnel, organisé et documenté. L'application peut être ouverte directement dans un navigateur et permet de gérer des tâches avec persistance locale. Le dépôt contient un historique Git réaliste, des branches de fonctionnalités, des merges, un conflit résolu, un README professionnel, une documentation des commandes Git et des emplacements détaillés pour les captures d'écran.

[CAPTURE D'ÉCRAN ICI]
Description : Structure finale du projet dans l'explorateur de fichiers ou dans l'éditeur de code.

[CAPTURE D'ÉCRAN ICI]
Description : Code source JavaScript montrant la fonction de rendu des tâches et les métriques.

[CAPTURE D'ÉCRAN ICI]
Description : Historique des commits affiché dans GitHub ou avec `git log --oneline --graph --all`.

Conclusion générale

Ce projet démontre que Git et GitHub ne sont pas de simples outils techniques, mais des supports d'organisation. Grâce aux commits, il devient possible de comprendre l'évolution du travail. Grâce aux branches, chaque fonctionnalité peut être développée sans perturber la version stable. Grâce aux merges et aux pull requests, le binôme peut intégrer progressivement les contributions. Enfin, grâce à la résolution de conflit, les étudiants apprennent à transformer une situation bloquante en décision collaborative.

StudyBoard Collab reste volontairement modeste sur le plan fonctionnel, mais il remplit son objectif pédagogique. Il montre une structure professionnelle, une application utilisable, un historique cohérent et une documentation détaillée. Le travail réalisé correspond donc à une simulation complète d'un projet universitaire collaboratif mené sur plusieurs jours.

Bibliographie

Documentation officielle Git : <https://git-scm.com/doc>

Documentation GitHub Docs : <https://docs.github.com/>

Pro Git Book, Scott Chacon et Ben Straub, Apress.

Annexes

Annexe A - Captures d'écran à insérer

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : création repository GitHub. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : interface GitHub. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : terminal Git. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : git status. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : git add. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : git commit. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : git push. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : git pull. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : branches. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : merge. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : conflit Git. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : résolution conflit. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : historique commits. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : pull request. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : structure projet. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

[CAPTURE D'ÉCRAN ICI]

Description : Capture obligatoire : code source. Elle doit être remplacée par une image réelle lors de la mise en page finale si l'enseignant exige des preuves visuelles.

Annexe B - État final du dépôt

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Annexe C - Extraits de commandes terminal

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe pull
origin develop
Already up to date.
From C:\Users\8RABB\Desktop\New folder\StudyBoard-Collab-origin
* branch develop -> FETCH_HEAD
```

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe stash
push -m wip: note temporaire README
Saved working directory and index state On develop: wip: note temporaire README
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
```

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe stash
pop
On branch develop
Your branch is ahead of 'origin/develop' by 2 commits.
(use "git push" to publish your local commits)
```

```
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: README.md
```

```
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (1bba1352cdc5a96e4e3994fbc7f1dffa490fff)
```

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe
restore README.md
```

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe
restore css/styles.css
```

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe add
README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
```

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe reset
README.md
Unstaged changes after reset:
M README.md
```

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe
restore README.md
```

```
$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe switch
main
Your branch is up to date with 'origin/main'.
Switched to branch 'main'
```

```

$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe merge
--no-ff develop -m release: fusionner develop dans main
Merge made by the 'ort' strategy.
README.md | 78 +++++-
assets/.gitkeep | 0
css/styles.css | 313 ++++++
docs/captures.md | 87 ++++++
docs/commandes-git-detaillées.md | 434 ++++++
docs/conflit-resolution.md | 28 +++
docs/notes-de-reunion.md | 13 ++
docs/workflow-git.md | 13 ++
index.html | 135 ++++++
js/app.js | 156 ++++++
10 files changed, 1255 insertions(+), 2 deletions(-)
create mode 100644 assets/.gitkeep
create mode 100644 css/styles.css
create mode 100644 docs/captures.md
create mode 100644 docs/commandes-git-detaillées.md
create mode 100644 docs/conflit-resolution.md
create mode 100644 docs/notes-de-reunion.md
create mode 100644 docs/workflow-git.md
create mode 100644 index.html
create mode 100644 js/app.js

$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe push
origin main
To C:\Users\8RABB\Desktop\New folder\StudyBoard-Collab-origin.git
86728a9..1d2bba2 main -> main

$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe status
--short

$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe log
--oneline --graph --all --decorate --date=short
* 1d2bba2 (HEAD -> main, origin/main) release: fusionner develop dans main
|\
| * 89dd91f (develop) merge: integrer la documentation etudiant 2
| |\
| | * 7cecb18 (origin/feature-docs-student2) docs: ajouter une capture de pull request
| | /
| | * 3895802 (origin/develop) docs: enrichir le README professionnel
| | * 1e7e884 chore: supprimer le brouillon obsolete avec git rm
| | * 86843b3 merge: integrer feature-dashboard
| | \
| | * 609abe0 (feature-dashboard) feat: exposer l'etat du tableau de bord pour validation
| | /
| | * 09f2ba6 merge: resoudre le conflit entre develop et feature-auth
| | \
| | * 6f269b9 (feature-auth) feat: ajouter une connexion locale de demonstration
| | * | 8bba4d1 feat: preciser le message du tableau de bord
| | /
| | * f6b206e merge: integrer feature-ui dans develop
| | \
| | * 3ebb339 (feature-ui) style: ameliorer les etats visuels des taches
| | /
| | * cbad997 docs: documenter le workflow et les commandes Git
| | * aa808fe feat: implementer la gestion local

$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe branch
-a
develop
feature-auth
feature-dashboard
feature-ui
* main
remotes/origin/develop
remotes/origin/feature-docs-student2
remotes/origin/main

$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe remote
-v
origin C:\Users\8RABB\Desktop\New folder\StudyBoard-Collab-origin.git (fetch)
origin C:\Users\8RABB\Desktop\New folder\StudyBoard-Collab-origin.git (push)

$ C:\Program Files\Microsoft Visual
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe status
On branch main
Your branch is up to date with 'origin/main'.

```

nothing to commit, working tree clean

```
$ C:\Program Files\Microsoft Visual  
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe add -A  
warning: in the working copy of 'docs/historique-git-reel.txt', LF will be replaced by CRLF the next time Git  
touches it  
warning: in the working copy of 'docs/transcription-commandes-terminal.md', LF will be replaced by CRLF the  
next time Git touches it
```

```
$ C:\Program Files\Microsoft Visual  
Studio\18\Community\Common7\IDE\CommonExtensions\Microsoft\TeamFoundation\Team Explorer\Git\cmd\git.exe commit  
-m docs: ajouter les transcriptions terminal et historique Git  
[main 28425a4] docs: ajouter les transcriptions terminal et historique Git  
2 files changed, 714 insertions(+)  
create mode 100644 docs/historique-git-reel.txt  
create mode 100644 docs/transcription-commandes-terminal.md
```


Tableaux du rapport

Technologie	Utilisation	Justification
HTML5	Structure des pages	Sémantique claire, compatibilité navigateur et simplicité de déploiement.
CSS3	Design responsive	Permet une interface professionnelle sans dépendance externe.
JavaScript	Comportement dynamique	Gestion des tâches, filtres, métriques et stockage local.
Git	Versionnement	Historique fiable, branches, merges et résolution de conflit.
GitHub	Collaboration distante	Pull requests, visibilité du dépôt, revue et synchronisation.

Commit représentatif	Auteur	Explication
chore: initialiser le depot et les metadonnees	Med Khalil	Création du README initial, licence et .gitignore.
feat: ajouter la page d'accueil et les sections principales	Med Khalil	Ajout de la structure HTML principale.
feat: implementer la gestion locale des taches	Amen Alah	Ajout de la logique JavaScript et du stockage local.
merge: resoudre le conflit entre develop et feature-auth	Med Khalil	Fusion manuelle de deux modifications concurrentes.
release: fusionner develop dans main	Med Khalil	Préparation de la version finale stable.

Étudiant	Tâches	Branches concernées	Livrables
Med Khalil Milliti	Interface, CSS, connexion, résolution conflit	feature-ui, feature-auth	HTML, CSS, README
Amen Alah ELHaj	JavaScript, documentation Git, dashboard	feature-dashboard, feature-docs-student2	JS, docs, historique

Branche	Rôle	Exemple de commande
main	Version stable	git switch main
develop	Intégration	git switch develop
feature-ui	Interface	git switch -c feature-ui
feature-auth	Connexion	git merge feature-auth
feature-dashboard	Indicateurs	git push origin feature-dashboard

Difficulté	Cause	Conséquence
Conflit dans index.html	Modification parallèle de la même phrase	Merge interrompu jusqu'à résolution manuelle.

Difficulté	Cause	Conséquence
Remote GitHub non authentifié	Absence de compte connecté dans l'environnement	Utilisation d'un remote bare local pour exécuter push/pull.
Historique détaillé	Nombreuses étapes à conserver	Création d'une transcription terminal et d'un rapport structuré.